

## Session 5: Hypothesis Testing

Compiled by D K. Muriithi

### Hypothesis Testing in Data Science

**Hypothesis testing** is a statistical method to make decisions about data or validate assumptions. Here's what you need to know:

#### Core Concept

You start with two hypotheses:

- **Null Hypothesis ( $H_0$ ):** The default assumption (e.g., "there's no difference")
- **Alternative Hypothesis ( $H_1$ ):** What you're trying to prove (e.g., "there is a difference")

#### Key Steps

1. **Set significance level ( $\alpha$ )** — typically 0.05 (5% chance of rejecting  $H_0$  when it's true)
2. **Choose a test** — depends on your data type and question (t-test, chi-square, ANOVA, etc.)
3. **Calculate test statistic** — measure the evidence against  $H_0$
4. **Compute p-value** — probability of seeing this result if  $H_0$  is true
5. **Make a decision:**
  - If  $p\text{-value} < \alpha \rightarrow$  reject  $H_0$  (statistically significant result)
  - If  $p\text{-value} \geq \alpha \rightarrow$  fail to reject  $H_0$

#### Common Tests in Data Science

- **T-test** — compare means between two groups
- **Chi-square** — test relationships between categorical variables
- **ANOVA** — compare means across multiple groups
- **Correlation tests** — check if variables are related

### Install library

```
In [6]: #!pip install pingouin  
#!pip install shapiro
```

### Load Necessary Libraries

```
In [7]: import numpy as np  
import pandas as pd  
import scipy.stats as stats  
import os  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import scipy as sp
import pingouin as pg
from scipy.stats import shapiro
```

## Load required Dataset

```
In [8]: gss = pd.read_csv("GSSsubset.csv")
gss.head()
```

```
Out[8]:
```

|   | id | sex    | degree      | income   | marital  | age | height | weight | hrswrk |
|---|----|--------|-------------|----------|----------|-----|--------|--------|--------|
| 0 | 1  | MALE   | BACHELOR    | 60967.50 | DIVORCED | 53  | 72     | 190    | 60     |
| 1 | 2  | FEMALE | BACHELOR    | 60967.50 | MARRIED  | 26  | 60     | 97     | 40     |
| 2 | 4  | FEMALE | BACHELOR    | 10161.25 | MARRIED  | 56  | 68     | 160    | 20     |
| 3 | 14 | FEMALE | HIGH SCHOOL | 17551.25 | MARRIED  | 40  | 65     | 156    | 37     |
| 4 | 16 | MALE   | HIGH SCHOOL | 17551.25 | MARRIED  | 56  | 66     | 210    | 6      |

## One Sample T-test on Age

Null hypothesis: sample mean is significantly different from the population mean

Alternative hypothesis: no significant difference between sample and population mean.

```
In [9]: import numpy as np
from scipy import stats
```

```
In [10]: # Example: Perform One-Sample t-Test on Age( assume theoritical age = 30 years)
t_statistic, p_value = stats.ttest_1samp(gss["age"], 30)

# Output results
print("t-statistic:", t_statistic)
print("p-value:", p_value)
```

```
t-statistic: 34.79914719592315
p-value: 4.127386144551695e-174
```

```
In [11]: # Interpret result
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: sample mean is significantly different from")
else:
    print("Fail to reject the null hypothesis: no significant difference between sa")
```

Reject the null hypothesis: sample mean is significantly different from the population mean.

## Alternatively

```
In [12]: import pingouin as pg          # necessary library

age_data = gss['age'].dropna()      # Drop missing values in 'age' column

# Perform one-sample t-test against a population mean of 30
test_value = 30
one_sample_test = pg.ttest(age_data, test_value)
one_sample_test
```

```
Out[12]:
```

|               | T         | dof | alternative | p_val         | CI95           | cohen_d  | power | BF10       |
|---------------|-----------|-----|-------------|---------------|----------------|----------|-------|------------|
| <b>T_test</b> | 34.799147 | 993 | two-sided   | 4.127386e-174 | [43.67, 45.31] | 1.103762 | 1.0   | 1.284e+170 |

## One Sample T-test on Income

```
In [13]: import numpy as np
from scipy import stats

# Example: Perform One-Sample t-Test on Income
t_statistic, p_value = stats.ttest_1samp(gss["income"], 25000)

# Output results
print("t-statistic:", t_statistic)
print("p-value:", p_value)

# Interpret result
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: sample mean is significantly different from")
else:
    print("Fail to reject the null hypothesis: no significant difference between sa")
```

t-statistic: 10.799800232016514

p-value: 8.752187280090169e-26

Reject the null hypothesis: sample mean is significantly different from the population mean.

## Alternatively

```
In [14]: import pingouin as pg          # Necessary Library

income_data = gss['income'].dropna()      # Drop missing values in 'Income' column

# Perform one-sample t-test against a population mean of 25000
test_value = 25000
one_sample_test = pg.ttest(income_data, test_value)
one_sample_test
```

|               | T       | dof | alternative | p_val        | CI95                 | cohen_d  | power | BF10     |
|---------------|---------|-----|-------------|--------------|----------------------|----------|-------|----------|
| <b>T_test</b> | 10.7998 | 993 | two-sided   | 8.752187e-26 | [34727.28, 39047.16] | 0.342549 | 1.0   | 2.71e+22 |

## Two-Sample t-Test

### Independent Two-Sample t-Test (Age by Gender)

```
In [15]: import numpy as np
from scipy import stats

# Sample data: group A and group B
group_A = gss[gss["sex"] == "MALE"]["age"]
group_B = gss[gss["sex"] == "FEMALE"]['age']

# Perform two-sample t-test (equal variance assumed)
t_statistic, p_value = stats.ttest_ind(group_A, group_B, equal_var=True)

# Output results
print("t-statistic:", t_statistic)
print("p-value:", p_value)

# Interpret the result
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: the group means are significantly different.")
else:
    print("Fail to reject the null hypothesis: no significant difference between gr
```

t-statistic: 0.9526944573413797

p-value: 0.34097697352570455

Fail to reject the null hypothesis: no significant difference between group means.

## Alternatively

```
In [16]: import pingouin as pg # necessary library

# Separate age data by gender, dropping missing values
group_A = gss[gss['sex'] == 'MALE']['age'].dropna()
group_B = gss[gss['sex'] == 'FEMALE']['age'].dropna()

# Perform independent two-sample t-test (Welch's t-test)
two_sample_test = pg.ttest(group_A, group_B, paired=False)
two_sample_test
```

|               | T        | dof        | alternative | p_val    | CI95          | cohen_d  | power    | BF10  |
|---------------|----------|------------|-------------|----------|---------------|----------|----------|-------|
| <b>T_test</b> | 0.952437 | 989.144792 | two-sided   | 0.341108 | [-0.84, 2.43] | 0.060448 | 0.158478 | 0.111 |

## Independent Two-Sample t-Test (Income by Gender)

```
In [17]: import numpy as np
from scipy import stats

# Sample data: group A and group B
group_A = gss[gss["sex"] == "MALE"]["income"]
group_B = gss[gss["sex"] == "FEMALE"]['income']

# Perform two-sample t-test (equal variance assumed)
t_statistic, p_value = stats.ttest_ind(group_A, group_B, equal_var=True)

# Output results
print("t-statistic:", t_statistic)
print("p-value:", p_value)

# Interpret the result
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: the group means are significantly different.")
else:
    print("Fail to reject the null hypothesis: no significant difference between gr
```

t-statistic: 8.863375699222146

p-value: 3.534034483650988e-18

Reject the null hypothesis: the group means are significantly different.

## Alternatively

```
In [18]: import pingouin as pg                # necessary library

# Separate age data by gender, dropping missing values
group_A = gss[gss['sex'] == 'MALE']['income'].dropna()
group_B = gss[gss['sex'] == 'FEMALE']['income'].dropna()

# Perform independent two-sample t-test (Welch's t-test)
two_sample_test = pg.ttest(group_A, group_B, paired=False)
two_sample_test
```

```
Out[18]:
```

|        | T        | dof        | alternative | p_val        | CI95                 | cohen_d  | power | BF10      |
|--------|----------|------------|-------------|--------------|----------------------|----------|-------|-----------|
| T_test | 8.950355 | 825.422137 | two-sided   | 2.304728e-18 | [14673.48, 22917.25] | 0.562372 | 1.0   | 2.429e+11 |



## Paired t-Test

```
In [19]: import numpy as np
from scipy import stats

# Example: test scores before and after an intervention for the same students
before = [72, 75, 78, 71, 69, 74, 77, 73, 76, 70]
```

```

after = [75, 78, 80, 74, 72, 77, 79, 75, 78, 73]

# Perform paired t-test
t_statistic, p_value = stats.ttest_rel(before, after)

# Output results
print("t-statistic:", t_statistic)
print("p-value:", p_value)

# Interpret the result
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: there is a significant difference between th
else:
    print("Fail to reject the null hypothesis: no significant difference between th

```

t-statistic: -15.921683328090657

p-value: 6.713856195677205e-08

Reject the null hypothesis: there is a significant difference between the paired samples.

## Alternatively

```

In [20]: import pingouin as pg                # necessary library

# Perform paired t-test
pg.ttest(before, after, paired = True)

```

```

Out[20]:

```

|               | T          | dof | alternative | p_val        | CI95           | cohen_d  | power    | BF10     |
|---------------|------------|-----|-------------|--------------|----------------|----------|----------|----------|
| <b>T_test</b> | -15.921683 | 9   | two-sided   | 6.713856e-08 | [-2.97, -2.23] | 0.908575 | 0.725205 | 1.65e+05 |

## ✓ Two-Sample t-Test Using statsmodels

```

In [21]: import numpy as np
import statsmodels.api as sm
from statsmodels.stats.weightstats import ttest_ind

# Example: Malaria parasite load (mean parasite count per µL) in two counties
county_A = [230, 250, 245, 270, 255, 240, 260]
county_B = [210, 215, 220, 200, 225, 230, 205]

# Perform two-sample t-test (equal variance assumed by default)
t_stat, p_value, df = ttest_ind(county_A, county_B, usevar='pooled') # usevar='une

# Output results
print("t-statistic:", t_stat)
print("p-value:", p_value)
print("degrees of freedom:", df)

# Interpret the result
alpha = 0.05
if p_value < alpha:

```

```
print("Reject null hypothesis: Mean parasite loads are significantly different  
else:  
print("Fail to reject null hypothesis: No significant difference in parasite lo
```

t-statistic: 5.422176684690384

p-value: 0.00015444433003956295

degrees of freedom: 12.0

Reject null hypothesis: Mean parasite loads are significantly different between the counties.

## Chi-Square Test for Independence

This test is used to determine whether there is a significant association between two categorical variables

```
In [22]: import pandas as pd  
from scipy import stats  
  
# Create a Contingency Table (Gender vs Marital Status)  
contingency_table = pd.crosstab(gss["sex"], gss["marital"])  
  
# perform chi-square test  
chi2, p, dof, expected = stats.chi2_contingency(contingency_table)  
  
print("Chi-Square Test Result (Gender vs Marital Status):")  
print("Chi2:", chi2, "p-value:", p, "Degrees of Freedom:", dof)  
print("p-value:", p)  
print("Degrees of Freedom:", dof)  
  
# Interpret the result  
alpha = 0.05  
if p < alpha:  
    print("Reject the null hypothesis: there is a significant association between t  
else:  
    print("Fail to reject the null hypothesis: no significant association between t
```

Chi-Square Test Result (Gender vs Marital Status):

Chi2: 6.211308199614711 p-value: 0.18391363117203863 Degrees of Freedom: 4

p-value: 0.18391363117203863

Degrees of Freedom: 4

Fail to reject the null hypothesis: no significant association between the variable s.

## Alternative

Perform Chi-square test for independence between 'sex' and 'marital'

```
In [23]: import pingouin as pg # necessary library  
  
expected, observed, stats = pg.chi2_independence(data=gss, x='sex', y='marital')
```

In [24]: expected

Out[24]:

|     | marital | DIVORCED | MARRIED | NEVER MARRIED | SEPARATED | WIDOWED |
|-----|---------|----------|---------|---------------|-----------|---------|
| sex |         |          |         |               |           |         |

|        |           |            |  |            |           |           |
|--------|-----------|------------|--|------------|-----------|-----------|
| FEMALE | 84.269618 | 235.171026 |  | 131.793763 | 18.127767 | 17.637827 |
| MALE   | 87.730382 | 244.828974 |  | 137.206237 | 18.872233 | 18.362173 |

In [25]: observed

Out[25]:

|     | marital | DIVORCED | MARRIED | NEVER MARRIED | SEPARATED | WIDOWED |
|-----|---------|----------|---------|---------------|-----------|---------|
| sex |         |          |         |               |           |         |

|        |    |     |  |     |    |    |
|--------|----|-----|--|-----|----|----|
| FEMALE | 85 | 220 |  | 140 | 19 | 23 |
| MALE   | 87 | 260 |  | 129 | 18 | 13 |

In [26]: stats

Out[26]:

|   | test               | lambda    | chi2     | dof | pval     | cramer   | power    |
|---|--------------------|-----------|----------|-----|----------|----------|----------|
| 0 | pearson            | 1.000000  | 6.211308 | 4.0 | 0.183914 | 0.079049 | 0.485258 |
| 1 | cressie-read       | 0.666667  | 6.218386 | 4.0 | 0.183422 | 0.079094 | 0.485763 |
| 2 | log-likelihood     | 0.000000  | 6.249637 | 4.0 | 0.181265 | 0.079293 | 0.487987 |
| 3 | freeman-tukey      | -0.500000 | 6.288514 | 4.0 | 0.178613 | 0.079539 | 0.490748 |
| 4 | mod-log-likelihood | -1.000000 | 6.341226 | 4.0 | 0.175073 | 0.079872 | 0.494482 |
| 5 | neyman             | -2.000000 | 6.491066 | 4.0 | 0.165354 | 0.080810 | 0.505024 |

## Fisher's Exact Test (For Small Sample Sizes)

In [27]:

```
if np.any(expected < 5): # Only run Fisher's test if expected values < 5
    fisher_test = stats.fisher_exact(contingency_table)
    print("Fisher's Exact Test Result:", fisher_test)
```

## One-Sample Z-Test with scipy.stats

In [28]:

```
import numpy as np
from scipy.stats import norm

# Example: test if sample mean differs from a population mean
sample_data = [101, 98, 103, 100, 99, 102, 97, 100, 104, 99]
population_mean = 100
population_std = 2 # known population standard deviation
n = len(sample_data)
```

```

# Calculate sample mean
sample_mean = np.mean(sample_data)

# Compute z-statistic
z_stat = (sample_mean - population_mean) / (population_std / np.sqrt(n))

# Compute two-tailed p-value
p_value = 2 * (1 - norm.cdf(abs(z_stat)))

# Output results
print("Z-statistic:", z_stat)
print("p-value:", p_value)

# Interpret result
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: sample mean is significantly different from")
else:
    print("Fail to reject the null hypothesis: no significant difference.")

```

Z-statistic: 0.4743416490252524

p-value: 0.6352562959972516

Fail to reject the null hypothesis: no significant difference.

## One-Sample Z-Test with statsmodels

In [29]:

```

import numpy as np
from statsmodels.stats.weightstats import ztest

# Example data: sample values
sample_data = [101, 98, 103, 100, 99, 102, 97, 100, 104, 99]
population_mean = 100

# Perform one-sample Z-test
z_stat, p_value = ztest(sample_data, value=population_mean)

# Output results
print("Z-statistic:", z_stat)
print("p-value:", p_value)

# Interpretation
alpha = 0.05
if p_value < alpha:
    print("Reject null hypothesis: sample mean significantly differs from populatio")
else:
    print("Fail to reject null hypothesis: no significant difference.")

```

Z-statistic: 0.42857142857142455

p-value: 0.6682351417952523

Fail to reject null hypothesis: no significant difference.

## Two-Sample Z-Test with statsmodels

```
In [30]: from statsmodels.stats.weightstats import ztest

# Two independent samples
group1 = [105, 102, 100, 98, 97, 103, 99, 101]
group2 = [95, 96, 92, 94, 93, 91, 97, 95]

# Perform two-sample Z-test
z_stat, p_value = ztest(group1, group2, value=0, alternative='two-sided')

print("Z-statistic:", z_stat)
print("p-value:", p_value)

# Interpretation
if p_value < alpha:
    print("Reject null hypothesis: group means are significantly different.")
else:
    print("Fail to reject null hypothesis: no significant difference.")
```

Z-statistic: 5.4812812776251905

p-value: 4.222565249683579e-08

Reject null hypothesis: group means are significantly different.

## Assignment: Apply Hypothesis Tests on a New Dataset

```
In [31]: import seaborn as sns
cars = sns.load_dataset("mpg").dropna()
cars.head()
```

Out[31]:

|   | mpg  | cylinders | displacement | horsepower | weight | acceleration | model_year | origin |
|---|------|-----------|--------------|------------|--------|--------------|------------|--------|
| 0 | 18.0 | 8         | 307.0        | 130.0      | 3504   | 12.0         | 70         | usa    |
| 1 | 15.0 | 8         | 350.0        | 165.0      | 3693   | 11.5         | 70         | usa    |
| 2 | 18.0 | 8         | 318.0        | 150.0      | 3436   | 11.0         | 70         | usa    |
| 3 | 16.0 | 8         | 304.0        | 150.0      | 3433   | 12.0         | 70         | usa    |
| 4 | 17.0 | 8         | 302.0        | 140.0      | 3449   | 10.5         | 70         | usa    |

## One-Sample t-Test on MPG (Miles Per Gallon)

```
In [32]: # Drop missing values in 'age' column
mpg_data = cars['mpg'].dropna()

# Perform one-sample t-test against a population mean of 30
```

```
test_value = 25
one_sample_test = pg.ttest(mpg_data, test_value)
one_sample_test
```

```
Out[32]:
```

|               | T         | dof | alternative | p_val    | CI95           | cohen_d  | power    | BF10    |
|---------------|-----------|-----|-------------|----------|----------------|----------|----------|---------|
| <b>T_test</b> | -3.942244 | 391 | two-sided   | 0.000096 | [22.67, 24.22] | 0.199113 | 0.975728 | 109.417 |

## Independent Two-Sample t-Test (MPG by Origin)

```
In [54]: import pandas as pd
import scipy.stats

t_test_mpg = scipy.stats.ttest_ind(usa_mpg, europe_mpg, equal_var=False)
t_test_mpg

# Extract the values directly from the t_test_mpg object
results_table = pd.DataFrame({
    'Test Statistic': [f"{t_test_mpg.statistic:.3f}"],
    'p-value': [f"{t_test_mpg.pvalue:.3e}"],
    'Degrees of Freedom': [f"{t_test_mpg.df:.2f}"]
})

display(results_table)
```

|          | Test Statistic | p-value   | Degrees of Freedom |
|----------|----------------|-----------|--------------------|
| <b>0</b> | -8.431         | 1.930e-13 | 105.32             |

## Alternatively

```
In [55]: pg.ttest(usa_mpg, europe_mpg, paired = False, alternative = "two-sided")
```

```
Out[55]:
```

|               | T         | dof        | alternative | p_val        | CI95           | cohen_d  | power | BF10      |
|---------------|-----------|------------|-------------|--------------|----------------|----------|-------|-----------|
| <b>T_test</b> | -8.431121 | 105.320342 | two-sided   | 1.929839e-13 | [-9.35, -5.79] | 1.169797 | 1.0   | 3.799e+12 |

## Chi-Square Test for Independence (Cylinders vs Origin)

```
In [56]: from scipy.stats import chi2_contingency

contingency_table_cyl = pd.crosstab(cars["origin"], cars["cylinders"])

chi2_cyl, p_cyl, dof_cyl, expected_cyl = chi2_contingency(contingency_table_cyl)

print("Chi-Square Test (Cylinders vs Origin):")
print(f"Chi2 = {chi2_cyl:.4f}")
print(f"p-value = {p_cyl:.4f}")
print(f"Degrees of Freedom = {dof_cyl}")
```

Chi-Square Test (Cylinders vs Origin):  
Chi2 = 180.7187  
p-value = 0.0000  
Degrees of Freedom = 8

## Alternative Chi-Square Test

```
In [57]: import pandas as pd
import pingouin as pg
import warnings
warnings.simplefilter(action='ignore')

expected, observed, stats = pg.chi2_independence(
    data=cars,
    x='origin',
    y='cylinders'
)

print("Observed Frequencies")
print(observed)

print("\nExpected Frequencies")
print(expected)

print("\nChi-Square Results")
print(stats)
```

```
Observed Frequencies
cylinders 3  4  5  6  8
origin
europe    0 61  3  4  0
japan     4 69  0  6  0
usa       0 69  0 73 103

Expected Frequencies
cylinders      3      4      5      6      8
origin
europe    0.693878 34.520408 0.520408 14.397959 17.867347
japan     0.806122 40.104592 0.604592 16.727041 20.757653
usa       2.500000 124.375000 1.875000 51.875000 64.375000

Chi-Square Results
      test      lambda      chi2  dof      pval      cramer \
0      pearson  1.000000 180.718694  8.0  7.272757e-35  0.480113
1      cressie-read  0.666667 182.529131  8.0  3.029776e-35  0.482512
2      log-likelihood  0.000000 210.502643  8.0  3.898520e-41  0.518168
3      freeman-tukey -0.500000      NaN  8.0      NaN      NaN
4  mod-log-likelihood -1.000000      inf  8.0  0.000000e+00      inf
5      neyman -2.000000      NaN  8.0      NaN      NaN

      power
0      1.0
1      1.0
2      1.0
3      NaN
4      NaN
5      NaN
```

View the Chi-Square Statistics and P-value

```
In [58]: observed

Out[58]: cylinders 3  4  5  6  8
          origin
          -----
europe    0 61  3  4  0
japan     4 69  0  6  0
usa       0 69  0 73 103

In [59]: expected
```

```
Out[59]:
```

|        | cylinders | 3          | 4        | 5         | 6         | 8 |
|--------|-----------|------------|----------|-----------|-----------|---|
| origin |           |            |          |           |           |   |
| europa | 0.693878  | 34.520408  | 0.520408 | 14.397959 | 17.867347 |   |
| japan  | 0.806122  | 40.104592  | 0.604592 | 16.727041 | 20.757653 |   |
| usa    | 2.500000  | 124.375000 | 1.875000 | 51.875000 | 64.375000 |   |

```
In [40]: stats
```

```
Out[40]:
```

|   | test               | lambda    | chi2       | dof | pval         | cramer   | power |
|---|--------------------|-----------|------------|-----|--------------|----------|-------|
| 0 | pearson            | 1.000000  | 180.718694 | 8.0 | 7.272757e-35 | 0.480113 | 1.0   |
| 1 | cressie-read       | 0.666667  | 182.529131 | 8.0 | 3.029776e-35 | 0.482512 | 1.0   |
| 2 | log-likelihood     | 0.000000  | 210.502643 | 8.0 | 3.898520e-41 | 0.518168 | 1.0   |
| 3 | freeman-tukey      | -0.500000 | NaN        | 8.0 | NaN          | NaN      | NaN   |
| 4 | mod-log-likelihood | -1.000000 | inf        | 8.0 | 0.000000e+00 | inf      | NaN   |
| 5 | neyman             | -2.000000 | NaN        | 8.0 | NaN          | NaN      | NaN   |

## Fishers Exact Test

```
In [41]: # Create a Contingency Table (Gender vs Marital Status)
contingency_table = pd.crosstab(cars["origin"], cars["cylinders"])
contingency_table
```

```
Out[41]:
```

|        | cylinders | 3  | 4 | 5  | 6   | 8 |
|--------|-----------|----|---|----|-----|---|
| origin |           |    |   |    |     |   |
| europa | 0         | 61 | 3 | 4  | 0   |   |
| japan  | 4         | 69 | 0 | 6  | 0   |   |
| usa    | 0         | 69 | 0 | 73 | 103 |   |

```
In [42]: # Convert the DataFrame to a NumPy array
contingency_table = contingency_table.to_numpy()

# Check for zero counts in the contingency table
if np.any(contingency_table == 0):
    print("Contingency table contains zero counts. Fisher's Exact Test may not be a
else:
    # Perform Fisher's Exact Test
    odds_ratio, p_value = fisher_exact(contingency_table)
    print(f"Fisher's Exact Test Result: Odds Ratio = {odds_ratio}, p-value = {p_val
```

Contingency table contains zero counts. Fisher's Exact Test may not be appropriate.

# Homework

## Exercise A: Import a Dataset

- Download a dataset (customerData.csv)
- Load it into R using "rio" package
- Print first few rows and check structure

## Exercise B: One-Sample T-Test

- Choose a numeric variable (e.g., income, spend etc)
- Define a hypothesized population mean
- Perform a one-sample t-test and interpret the result

## Exercise C: Two-Sample T-Test

- Split your data into two groups (e.g., male/female)
- Perform a two-sample t-test
- Report whether the difference is statistically significant

## Exercise D: Chi-Square Test

- Select two categorical variables (e.g., occupation vs payment\_type )
- Build a contingency table
- Perform a chi-square test for independence
- Interpret the result

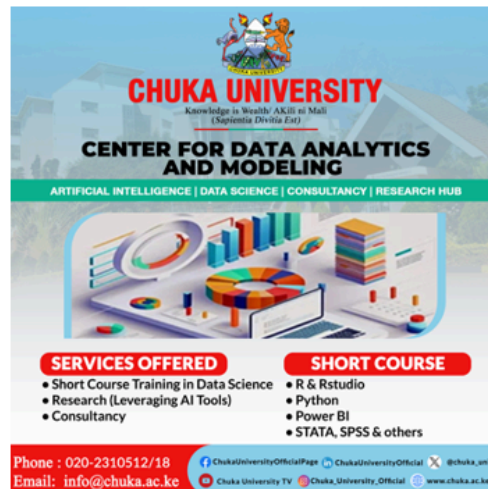
## Exercise E: Write a Summary

- Summarize your findings clearly in plain language
- Include:
  - What question you were answering
  - Which test you used

- Whether the result was significant
- What conclusion can be drawn

## Additional Resources

- R Documentation for stats : <https://stat.ethz.ch/R-manual/R-devel/library/stats/>
- Statistical Tests in R Cheat Sheet : <https://posit.co/download/rstudio-cheatsheets>
- Practical Statistics for Data Scientists : Book by Peter Bruce & Andrew Bruce
- Introduction to Statistical Learning (ISLR) : Free book with R labs – <http://faculty.marshall.usc.edu/gareth-james/ISL/>



<https://cdam.chuka.ac.ke>

**Thank you**